

Class 23 Linux File System Architecture & Logs For SOC

Linux File System Structure

The **Linux file system** is a way of organizing and storing files in a **single hierarchical tree structure**.

- It starts from the **Root Directory (/)**, which is the top-level directory.
- All files and folders are organized under this root.
- Unlike Windows, which has separate drives like **C:**, **D:**, **E:**, Linux does **not** use multiple drive letters.
- Everything in Linux (files, folders, devices, etc.) is accessed through the **/ root directory**.

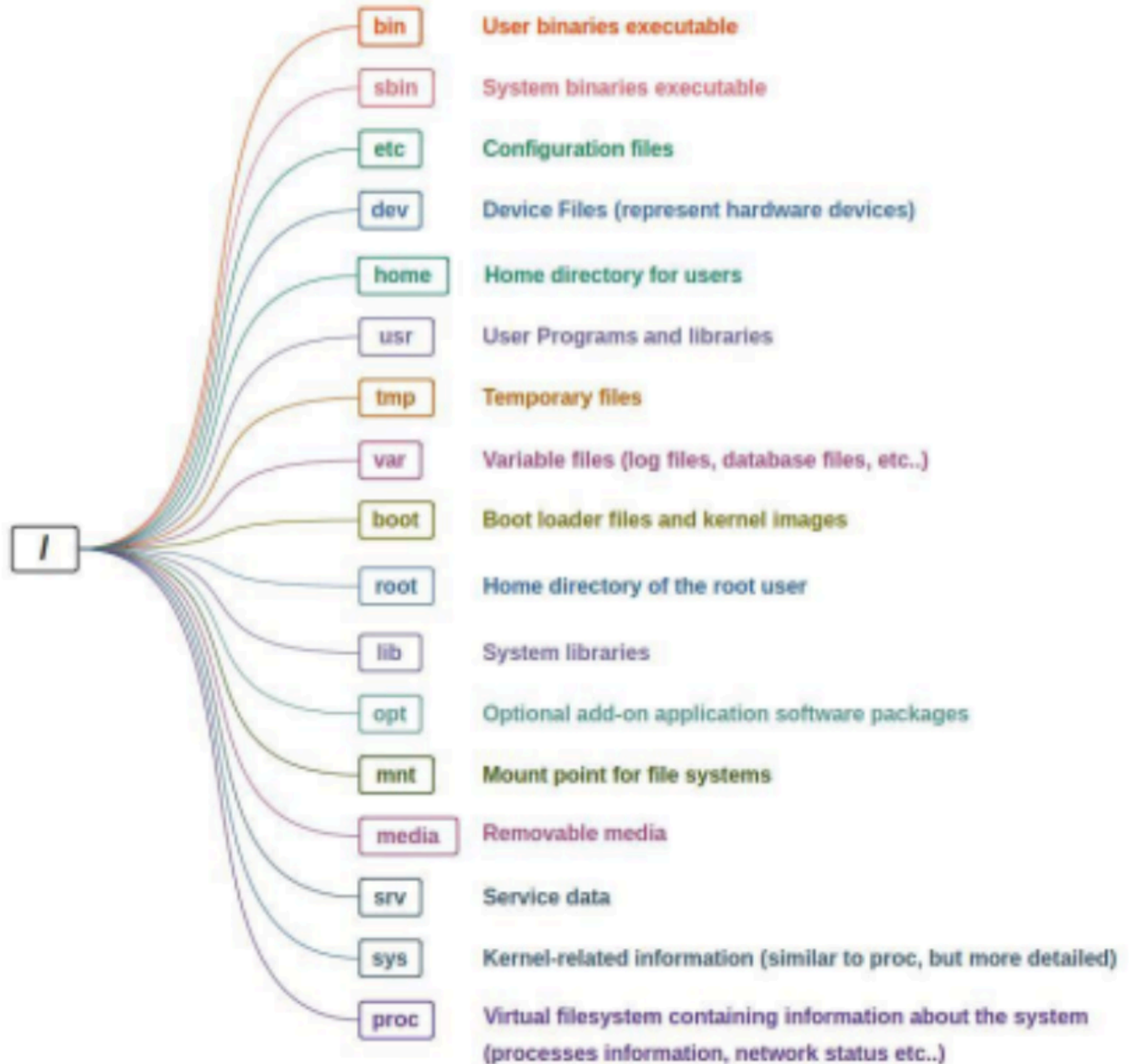
Example:

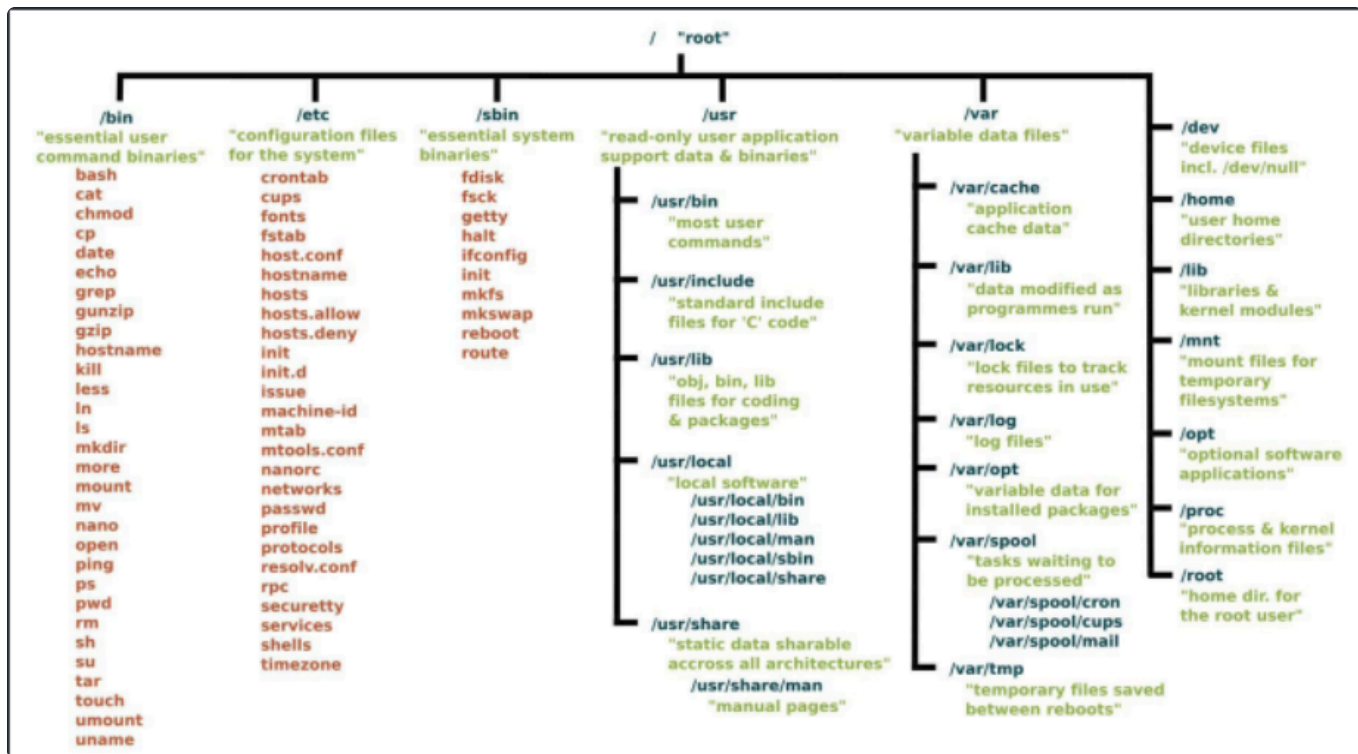
```
/
├── home
├── etc
├── bin
├── usr
└── var
```

In simple words:

The Linux file system is a **single tree-like structure** where **everything begins from the root directory (/)**.

LINUX FILE SYSTEM STRUCTURE





Linux Permissions

In Linux, everything is a file, and every file has an owner and a group. Permissions define how the Owner, the Group, and Everyone Else can interact with those files.

Permission	Symbol	File Meaning	Directory Meaning
Read	r	View the file contents	List the file inside
Write	w	Modify the file contents	Create, delete, or rename file inside
Execute	x	Run the file as a program	Enter the directory

Note: To delete a file, you actually need Write and execute permissions on the parent directory, not necessarily the file itself

When you run `ls -l`, you see a string like `-rwxr-xr--`. Let's dissect that:

1. The First Character: Indicates the file type (- for regular file, d for directory, l for symbolic link).
2. Sets of Three: The next nine characters are divided into three triplets:

- User (u): The individual who owns the file.
- Group (g): Members of the group assigned to the file.
- Others (o): Everyone else on the system.

Example: `-rwxr-xr--`

- User: `rwx` (Read, Write, Execute)
- Group: `r-x` (Read, Execute)
- Others: `r--` (Read only)

Numeric (Octal) Notation

System admins rarely type out `rwx`. Instead, we use a base-8 system. Each permission is assigned a value:

- Read (r) = 4
- Write (w) = 2
- Execute (x) = 1
- None (-) = 0

To get the number for a triplet, you just add them up.

$$4 (r) + 2 (w) + 1 (x) = 7$$

Total	Permissions	Description
7	<code>rwx</code>	Full control
6	<code>rw-</code>	Read and write
5	<code>r-x</code>	Read and execute
4	<code>r--</code>	Read-only
0	<code>---</code>	No access

Common command: `chmod 644 file.txt` sets the file to Read/write for owner, and Read-only for everyone else

Linux `/var/log`

`/var/log` is the **main logging directory** in Linux. It stores log files that record system and application activities.

It contains:

- **Authentication logs** – Login attempts and user authentication.
- **System logs** – System events, errors, and warnings.
- **Application logs** – Logs generated by installed applications.

Why SOC (Security Operations Center) cares:

- During a security investigation, analysts examine log files in `/var/log` to detect suspicious activities, identify attacks, troubleshoot issues, and perform forensic analysis.

In simple words:

`/var/log` is the **central location for Linux log files**, making it one of the most important directories for system monitoring and cybersecurity investigations.



`/var/log/auth.log` (Most Important)

Stores **authentication-related logs**.

Contains:

- SSH login attempts
- User authentication (successful/failed)
- `sudo` command usage

Why important: Used to detect unauthorized access and login attempts.

`/var/log/syslog`

Stores **general system logs**.

Contains:

- System events
- Service messages
- Errors and warnings

Why important: Helps troubleshoot system issues.

`/var/log/kern.log`

Stores **kernel logs**.

Contains:

- Hardware events
- Driver messages

- Kernel errors

Why important: Useful for diagnosing hardware and kernel problems.

/var/log/secure

Used in **Red Hat, CentOS, and RHEL.**

Contains:

- Authentication logs
- SSH logins
- `sudo` activity

Note: Similar to `/var/log/auth.log` on Debian/Ubuntu.

/var/log/messages

Stores **general system activity.**

Contains:

- System messages
- Service logs
- Boot information

Why important: Commonly used for troubleshooting on Red Hat-based systems.

/var/log/apache2/access.log

Stores **Apache web server access logs.**

Contains:

- Visitor IP addresses
- Requested URLs
- HTTP status codes
- Access timestamps

Why important: Used to analyze website traffic and detect suspicious requests.

`/var/log/apache2/error.log`

Stores **Apache web server error logs**.
Contains:

- Server errors
- Application errors
- Configuration issues

Why important: Helps identify website problems and attack attempts.

Quick Summary

Log File	Purpose
<code>/var/log/auth.log</code>	SSH logins, authentication, <code>sudo</code> activity
<code>/var/log/syslog</code>	General system logs
<code>/var/log/kern.log</code>	Kernel and hardware logs
<code>/var/log/secure</code>	Authentication logs (Red Hat/CentOS)
<code>/var/log/messages</code>	General system activity (Red Hat/CentOS)
<code>/var/log/apache2/access.log</code>	Website access logs
<code>/var/log/apache2/error.log</code>	Apache server error logs

Linux Authentication Logs

Authentication logs record **user login and authentication activities** on a Linux system.

They track:

- **User logins**
- **SSH access**
- **Failed login attempts**
- `sudo` command usage

Main Authentication Log Files

Operating System	Log File
Ubuntu/Debian	/var/log/auth.log
RHEL/CentOS	/var/log/secure

Example Log

```
Jul 10 10:22:11 server sshd[2345]: Accepted password for admin from 192.168.1.10
```

Explanation:

- **Jul 10 10:22:11** → Date and time of the event.
- **server** → Hostname of the Linux system.
- **sshd[2345]** → SSH daemon process (Process ID: 2345).
- **Accepted password** → Login was successful.
- **admin** → Username that logged in.
- **192.168.1.10** → IP address of the user who connected.

Why is it Important for SOC?

Security analysts use authentication logs to:

- Detect unauthorized login attempts.
- Identify brute-force attacks.
- Monitor SSH access.
- Track `sudo` usage and privilege escalation.

In simple words:

Authentication logs keep a record of **who logged in, when they logged in, from where they logged in, and whether the login was successful or failed.**

Linux SSH Logs

SSH (Secure Shell) is a secure network protocol used to **remotely access and manage Linux systems** over an encrypted connection.

- Provides **remote Linux access**
- Uses **Port 22** by default
- Encrypts communication between client and server

Why Attackers Love SSH

Attackers often target SSH because it allows them to:

- **Remote control** – Access and control a compromised Linux system.
- **Persistence** – Maintain access by adding SSH keys or creating user accounts.
- **Lateral movement** – Move from one compromised system to other systems within the network.

Where Are SSH Logs Stored?

Operating System	Log File
Ubuntu/Debian	/var/log/auth.log
RHEL/CentOS	/var/log/secure

Example SSH Log

```
Jul 10 10:22:11 server sshd[2345]: Accepted password for admin from 192.168.1.10
```

Explanation:

- **Jul 10 10:22:11** → Date and time
- **server** → Hostname
- **sshd[2345]** → SSH service (Process ID 2345)
- **Accepted password** → Successful login
- **admin** → Username
- **192.168.1.10** → Source IP address

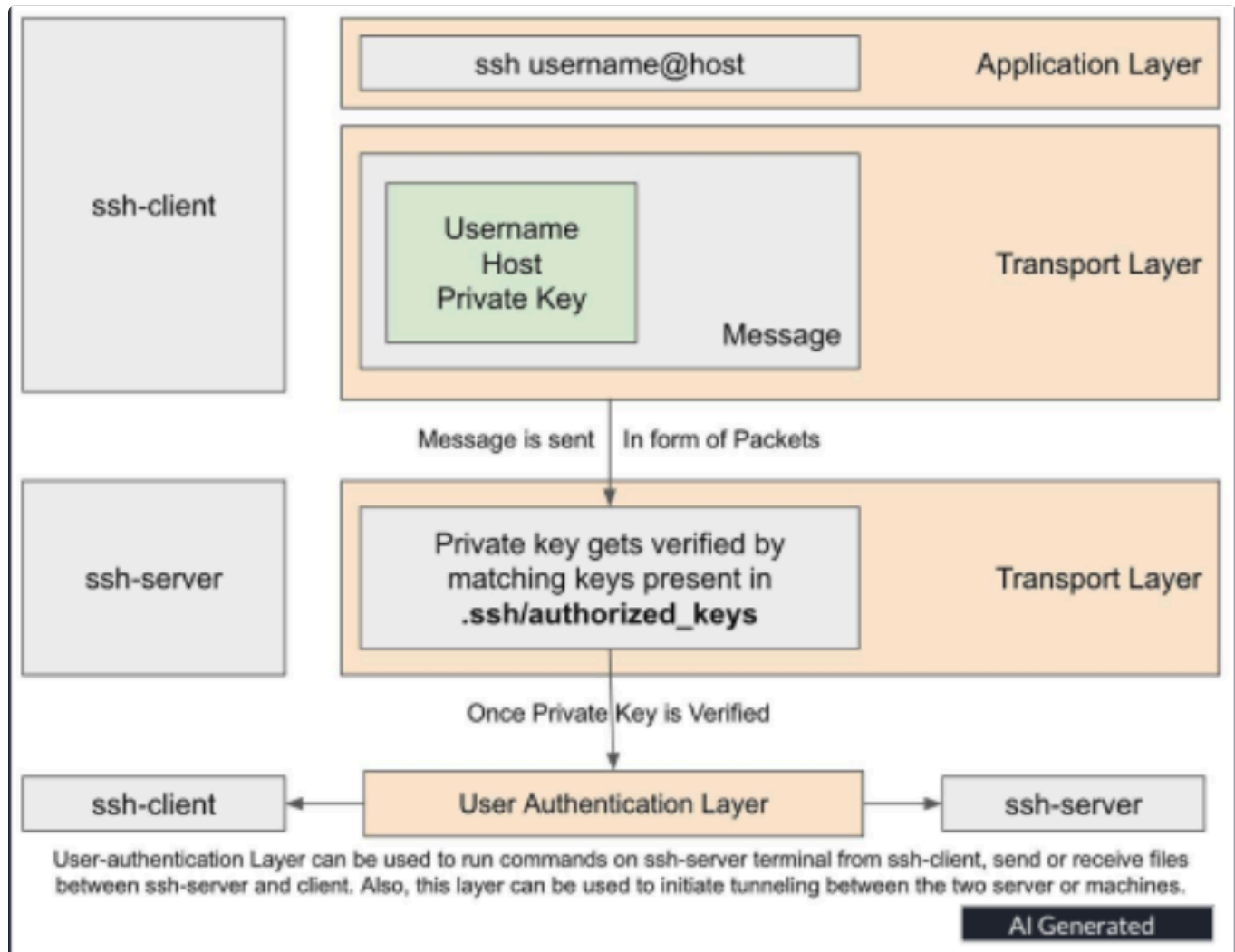
Why SOC Cares

SOC analysts monitor SSH logs to:

- Detect unauthorized or suspicious logins.
- Identify brute-force attacks.
- Track remote administrator access.
- Investigate security incidents and lateral movement.

In simple words:

SSH logs record **who connected** to a Linux system remotely, **when they connected**, **from which IP address**, and **whether the login succeeded or failed**.



Linux SSH Components

ssh

- **Client command** used to connect to a remote Linux system.
- Example:

```
ssh user@192.168.1.10
```

sshd

- **SSH daemon (server process)** that listens for incoming SSH connections.

- Authenticates users and provides remote access.

Port 22

- **Default port** used by SSH.
 - Clients connect to this port unless it has been changed for security reasons.
-

SSH Authentication Methods

1. Password Authentication

- User logs in using a **username and password**.
- Simple to use but more vulnerable to brute-force attacks if weak passwords are used.

2. Key-Based Authentication

- A **more secure** authentication method.
 - Uses:
 - **Public Key** – Stored on the remote server.
 - **Private Key** – Kept securely on the user's device.
 - No password is sent over the network, making it much harder for attackers to compromise.
-

Why SOC Cares

Security analysts monitor SSH authentication to:

- Detect unauthorized login attempts.
- Identify brute-force attacks against password logins.
- Verify that secure **key-based authentication** is being used.
- Investigate suspicious remote access.

In simple words:

- **ssh** = Client used to connect to a remote Linux system.
- **sshd** = Server process that accepts SSH connections.
- **Port 22** = Default SSH port.

- **Password authentication** uses a username and password, while **key-based authentication** uses a public/private key pair and is more secure.

Linux SSH Authentication

1. Password Authentication (Basic Method)

- The user logs in using a **username and password**.
- The password is sent through an **encrypted SSH tunnel**.
- The server verifies the password against `/etc/shadow` (Linux password database).
- If the credentials match, access is granted.

Disadvantage:

- Vulnerable to **brute-force** and password-guessing attacks if weak passwords are used.

2. Public Key Authentication (Secure Method)

Uses **Asymmetric Encryption** such as **RSA** or **Ed25519**.

Setup

You need:

- **Private Key** – Kept secret on your computer.
- **Public Key** – Stored on the server in:

```
~/.ssh/authorized_keys
```

Authentication Process

1. The client tells the server which public key it wants to use.
2. The server creates a random **challenge** and encrypts it using the user's **public key**.
3. The client decrypts the challenge using its **private key** and creates a **digital signature**.
4. The server verifies the signature.

5. If the signature is valid, the user is authenticated.

Advantages:

- More secure than passwords.
- Resistant to brute-force attacks.
- Widely used in professional environments.

SSH Configuration File

The main SSH server configuration file is:

```
/etc/ssh/sshd_config
```

This file controls settings such as:

- SSH port number
- Password authentication (enable/disable)
- Public key authentication
- Root login permissions
- Allowed users

Quick Comparison

Password Authentication	Public Key Authentication
Uses username + password	Uses public/private key pair
Easier to set up	Requires key generation
Less secure	More secure
Vulnerable to brute-force attacks	Resistant to brute-force attacks
Common for beginners	Standard in enterprise environments

In simple words:

- **Password Authentication:** Log in with a username and password.

- **Public Key Authentication:** Log in using a private key, while the server verifies it using your public key stored in `~/.ssh/authorized_keys`.
- `/etc/ssh/sshd_config` is the main configuration file for the SSH server.

Why SSH is Important for SOC

Attackers often abuse **SSH** to maintain long-term access to compromised Linux systems.

A common technique is to:

- Add their own **SSH public key** to:

```
~/.ssh/authorized_keys
```

- This allows them to log in without knowing the user's password.
- This technique is called **persistence** because it helps attackers keep access even if passwords are changed.

Real SOC Investigation Flow

Scenario

A SOC analyst observes:

1. **Many failed SSH login attempts**
2. **One successful SSH login**
3. `sudo` commands executed shortly after

SOC Thinking

Many failed SSH logins

→ Possible **brute-force attack** (attacker trying many passwords).

↓

Successful SSH login

→ Possible **system compromise** (attacker guessed or obtained the correct credentials).

↓

`sudo` activity

→ Possible **privilege escalation** (attacker is attempting to gain or use administrator/root privileges).

Investigation Steps for SOC

- Check authentication logs (`/var/log/auth.log` or `/var/log/secure`).
- Identify the source IP address.
- Count failed and successful login attempts.
- Review `sudo` commands executed.
- Check for newly added SSH keys in:

```
~/.ssh/authorized_keys
```

- Determine whether the activity is legitimate or malicious.
-

Simple Flow Diagram

```
graph TD; A[Failed SSH Logins] --> B[Successful Login]; B --> C[sudo Commands]; C --> D[Possible Brute Force]; D --> E[System Compromise]; E --> F[Privilege Escalation];
```

In Simple Words

SOC analysts monitor SSH logs to detect attacks. A pattern of **multiple failed logins** → **one successful login** → **sudo usage** is a strong indicator of a possible **brute-force attack, system compromise, and privilege escalation**, which requires immediate investigation.

Final SOC Mindset

When investigating a Linux system, a SOC analyst should **not** stop at checking whether a login occurred.

❌ Don't Ask

- "Was there a login?"

✅ Ask These Questions

- **Who logged in?** → Which user account was used?
- **From where?** → What was the source IP address?
- **Using what method?** → Password authentication or SSH key authentication?
- **What happened after login?** → Which commands were executed? Were files modified or services accessed?
- **Was privilege escalation used?** → Did the user run `sudo`, switch to the root account, or gain higher privileges?

SOC Investigation Checklist

1. **Identify the user** who logged in.
2. **Verify the source IP address** and location.
3. **Check the authentication method** (password or SSH key).
4. **Review user activity** after login (commands, processes, file access).
5. **Look for privilege escalation** using `sudo` or root access.
6. **Determine if the activity is legitimate or malicious.**

SOC Investigation Flow

```
graph TD; A[Login Detected] --> B[Who logged in?]; B --> C[From which IP?]; C --> D[Authentication method?]; D --> E[ ];
```

What actions were performed?

↓

Was sudo/root used?

↓

Normal Activity or Security Incident?

In Simple Words

A SOC analyst's goal is **not just to detect a login**, but to understand the **entire attack chain**:

- **Who** logged in?
- **Where** did they connect from?
- **How** did they authenticate?
- **What** did they do after logging in?
- **Did they gain higher privileges?**

This approach helps identify unauthorized access, compromises, and potential attacks.